(https://github.com/digma-ai/digma) and Sprkl (https://sprkl.dev/) envision. The general direction is to make developers more productive by providing insights into code changes and their impacts on the application, using a variety of signals.

### 3.5.1 Instrumentation

We already covered the costs and benefits of instrumenting your code in detail in chapter 2 for logs, metrics, and traces. Depending on the programming languages you use (i.e., interpreted versus compiled), you're faced with different efforts to emit multiple signals. When possible, you should use auto-instrumentation, and it's usually a good idea to standardize on OpenTelemetry, to avoid reinstrumentation work when switching signal destinations. We will cover this in greater detail in chapter 4.

### 3.5.2 Proxy sources

In cases when you don't own the code but still want to get some signals out, you can use proxy sources. For example, consider the following cases in the context of scraping metrics with Prometheus:

- You want to add metrics from a message queue, like Kafka or an IoT device, to the mix, so you would run one of the many exporters (http://mng.bz/lWDj) that Prometheus can scrape.
- An alternative situation may arise where you do own the code (e.g., in the context of a batch job or Lambda function), which is a short-running process. Prometheus, on the other hand, assumes a long-running process it can scrape at any time; in this case, you'd use the Pushgateway (https://github.com/prometheus/pushgateway) to compensate for the impedance mismatch. You can learn more about this via the KubeCon EU 2022 talk "Fleeting Metrics: Monitoring Short-lived or Serverless Jobs" by Bartek Płotka and Saswata Mukherjee, available via YouTube (https://www.youtube.com/watch?v=rt4JiK995s8).

A proxy could also be an adapter (e.g., from a log file to `stdout`) to make certain signal types, like logs, compatible and usable in a cloud-native setup.

With this, we have completed the discussion of sources in our cloud-native setup, from compute to storage to network to your own code, and now we move on to the question of how the signals arrive at various destinations for consumption. In short, we move on to agents.

## Summary

- Sources are anything in the system under observation that emits telemetry signals, such as logs, metrics, traces, and profiles.
- If you own a source (e.g., the application code you write, running in a container), then you can instrument it; otherwise, you need to work around it, dealing with what the owner of the source emits.
- We can differentiate between sources types: compute, storage, and network.